⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Clause: 6 — Software Maintenance Process · **Register:** Document Register

> This document *is* the maintenance plan 6.1 asks for — its first version.

What the standard requires (Class C — every sub-clause of Clause 6)

REF	TITLE	APPLIES AT	OUTPUT
6.1	Establish software maintenance plan	A, B, C	Software maintenance plan
6.2.1.1	Monitor feedback	A, B, C	—
6.2.1.2	Document and evaluate feedback	A, B, C	Documented and evaluated feedback, with any problem recorded as a problem report
6.2.1.3	Evaluate problem report's effect on safety	A, B, C	Evaluation of each problem report's effect on safety, and whether a change is needed
6.2.2	Use software problem resolution process	A, B, C	—

REF	TITLE	APPLIES AT	OUTPUT
6.2.3	Analyse change requests	A, B, C	Analysis of each change request's effect on the organisation, released software and interfacing systems
6.2.4	Change request approval	A, B, C	Evaluated and approved change requests
6.2.5	Communicate to users and regulators	A, B, C	Identification of approved changes affecting released software, and the information given to users/regulators
6.3.1	Use established process to implement modification	A, B, C	—
6.3.2	Re-release modified software system	A, B, C	—

Clause 6 is uniformly all-classes since Amendment 1 moved 6.2.3 up from B/C — see the site's own Safety Class Applicability section.

6.1 — The plan

1. **Feedback arrives** through the contact form (§6.2.1.1) or is discovered directly during development (a test failure, a manual check against the standard).
2. **Feedback is logged and evaluated** (§6.2.1.2–6.2.1.3): a test failure is logged automatically in `tests/anomaly_log.csv`; feedback from the contact form is read and, if it describes a real defect, entered into the same process manually — there being no separate ticketing system for externally-reported issues at this project's size.
3. **The problem resolution process is used** (§6.2.2) — see 13 — for both paths, so a defect found by a test and a defect reported by a visitor are handled identically from that point on.

4. **Change requests are analysed and approved** (§6.2.3–6.2.4): for a solo-maintained project this is a single reviewer decision rather than a change control board, recorded in the git commit that makes the change — see 12 — Configuration Management Plan.
5. **Users and regulators are informed of changes** (§6.2.5): there is no regulator in this project's case; "users" are the site's visitors, who are not proactively notified of content changes beyond what the Edition 2 notice and version chip already state on the page itself (see 09 on the gap in formally dated releases this weakens).
6. **The established process is used to implement and re-release** (§6.3.1–6.3.2): the same lifecycle activities described in 02 — Software Development Plan apply to a maintenance change as to original development — there is no separate, lighter-weight "just a maintenance fix" path that skips testing.

6.2.1.1 — Feedback monitoring, concretely

- **The contact form** (`contact.html`) — explicitly invites "Report an error in the course content," verified by REQ-25/26 in 03.
- **The test suite itself** — a large share of feedback this project acts on is self-generated: a check written to catch a defect, run on a schedule of "every time something changes," is a feedback channel the standard doesn't name explicitly but that functions exactly like one.

6.2.3 — Real example of a change request analysis

The safety-class mapping correction described in 11 — Risk Management File is a worked example of 6.2.3 in practice: a change request ("Clause 7's class mapping looks wrong") was analysed for its effect on released software (the Learn page's filter, the deliverables list, and the quiz questions that referenced the old mapping), not just patched at the one spot it was noticed.

Gaps

- No separate change-request log exists outside git commit messages and `tests/anomaly_log.csv` — for a project of this size the two together cover 6.2.1–6.2.4 adequately, but neither is *labelled* as a change-request register, which a stricter reading of 6.2.4 might expect.
- §6.2.5's "communicate to users and regulators" has no active notification mechanism (an email list, an RSS feed, a changelog page) — only the passive, on-page version/edition information described in 09.